Analyse Numérique
Cours de Licence : SMA4 - SMI4 - SMP4
2024-2025

Chapitre 1: Analyse d'erreurs

Benmoussa Khalid

Faculté des Sciences Agadir, Université Ibn-Zohr

2024-2025

Motivations

L'analyse numérique est devenue un outil indispensable dans l'industrie de pointe, les simulations numériques permettent entre autres de :

- Réduire les coûts de prototypage,
- Avoir accès à des mesures physiques très difficiles à mesurer expérimentalement.

Pierre-Louis Lions (Médaillé Fields, 1994) :

> *En quelques décennies, les* ***simulations numériques*** *sont devenues un* ***outil privilégié*** *d'investigation dans les sciences et les technologies. Elles ont pour but de* ***reproduire par le calcul*** *le comportement d'un système décrit par un modèle très souvent constitué d'équations aux dérivées partielles. (. . .) L'essor des simulations numériques renforce donc la* ***nécessité de l'étude mathématiques (analyse) de ces équations et de leur résolution numérique****...*

Motivations

Ce que vous savez déjà faire analytiquement :

- Résolution de certaines EDO, ex : $2y' = -3xy \Rightarrow$ séparation des variables : $y = Ce^{-\frac{3}{4}x^2}$
- Dérivation, intégration,...

Pour ces exemples, il est possible de faire les calculs analytiquement, et donc d'obtenir une solution exacte, mais cela n'est pas toujours possible...

Ce que nous allons voir :

- Des méthodes numériques permettant de résoudre avec un ordinateur des problèmes mathématiques répandus dans la «vraie vie» (système linéaire, EDO, intégrales, interpolation,...)
- Analyser la pertinence, précision des algorithmes ainsi que le coût et la complexité de mise en oeuvre,
- Chercher à évaluer l'erreur commise par l'utilisation d'une méthode numérique.

Chapitre 1 : Analyse d'erreurs

On souhaite simuler des phénomènes physiques : obtenir une **solution approchée**, ou **approximation**, d'une solution exacte le plus souvent inconnue . . .

Plusieurs sources d'erreurs vont intervenir, à différents niveaux :

- **Erreur de modélisation :** par exemple en négligeant certains termes physiques,
- **Erreur de représentation en machine :** pour un ordinateur il est impossible de représenter de manières exactes des nombres tels que $\frac{1}{3}, \pi, \ldots \rightarrow$ on procédera à l'arrondi, ou troncature,
- **Erreur de troncature :** certaines opérations (log, exp, sin,cos) sont approchées par une **somme finie** d'opérations élémentaires (+,-,x,/). Cependant, le plus souvent, pour être exacte, la somme devrait être infinie... Ce dernier type d'erreur interviendra notamment lors de l'étude des **développements de Taylor.**

Quelques catastrophes dues à une mauvaise utilisation de l'outil numérique : `http://ta.twi.tudelft.nl/users/vuik/wi211/disasters.html`

Definition

"Erreur absolue" Soit x un nombre réel, et x^* une approximation de ce nombre. L'erreur absolue est définie par

$$E_a(x^*) = |x - x^*|$$

Remarque : c'est une mesure **quantitative**.

Definition

"Erreur relative" Soit x un nombre réel, et x^* une approximation de ce nombre. L'erreur relative est définie par

$$E_r(x^*) = \frac{|x - x^*|}{|x|} = \frac{E_a(x^*)}{|x|}$$

Remarque : c'est une mesure **qualitative**, si on multiplie par 100, on obtient l'erreur relative en %

❗ En pratique, x n'est pas connu !

On exhibera le plus souvent une **borne supérieure** de l'erreur absolue, notée Δx, *i.e*

$$E_a(x^*) = |x - x^*| \leq \Delta x$$

Remark

$$|x - x^*| \leq \Delta x \iff -\Delta x \leq x - x^* \leq \Delta x \iff x^* - \Delta x \leq x \leq x^* + \Delta x$$

Autrement dit, $x^* = x \pm \Delta x$, Δx représente ainsi l'incertitude.

Par abus de langage, **on confondra la borne Δx avec $E_a(x^*)$.**

Pour l'erreur relative E_r, on a alors

$$E_r(x^*) = \frac{|x - x^*|}{|x|} \leq \frac{\Delta x}{|x|}$$

et si Δx est suffisamment «petit», on dira que

$$E_r(x^*) \approx \frac{\Delta x}{|x^*|}$$

En conclusion, **sauf si l'on connaît à la fois x et x^***, on utilisera :

- Δx comme valeur pour l'erreur absolue,
- $\frac{\Delta x}{|x^*|}$ comme valeur pour l'erreur relative.

Definition

"Chiffres significatifs" Si l'erreur absolue vérifie $\Delta x \leq 0,5 \times 10^m$, alors le chiffre dans x^* correspondant à la m^e puissance de 10 est dit *significatif* et tous ceux à sa gauche (correspondant aux puissances de 10 supérieures à m) le sont aussi. On arrête le compte au dernier chiffre non nul.

attention

Si le chiffre correspondant à la m^e puissance de 10 dans x^* est nul ainsi que tous ceux à sa gauche, on dit qu'il n'y a aucun chiffre significatif.

Exemples :

- On veut approcher le nombre $x = \pi$ par le nombre $x^* = \frac{22}{7} = 3,142857$
  $\Delta x = |x - x^*| = |\pi - \frac{22}{7}| = 0.00126 = 0.126 * 10^{-2} \leq 0.5 * 10^{-2}$.
  le chiffre 4 est significatif et tous les chiffres à gauche sont aussi significatis
  3,142857
- On veut approcher le nombre $x = \pi$ par le nombre $3,1416$
  $\Delta x = |x - x^*| = |\pi - 3.1416| = 0.73 * 10^{-5} \leq 0.5 * 10^{-4}$.
  le chiffre 6 est significatif et tous les chiffres à gauche sont aussi significatis
  3,1416
- On a mesuré le poid d'une personne $m = 90.567Kg$, on nous assure que l'appareil utilisé est précis et que tous les chiffres affichés sont significatifs.
  donc $\Delta x < 0.5 * 10^{-3}$.
  En pratique, on donne $\Delta x = 0.5 * 10^{-3}$.

Inversement, on estime l'erreur absolue (Δx) à partir du nombre de chiffres significatifs de la manière suivante : • Si une approximation x^* possède n chiffres

significatifs, on commence à compter à partir du premier chiffre non nul à gauche, et l'erreur absolue est inférieure à 0,5 fois la puissance de 10 correspondant au dernier chiffre significatif.

attention

Si x^* possède n chiffres significatifs, cela ne veut pas dire que x^* possède n chiffres exacts ! Mais n chiffres dont on contrôle l'erreur.

Représentation en virgule flottante :
Soit x un nombre réel. La représentation en virgule flottante de x dans **la base** b sera

$$\begin{aligned} x = \pm m \times b^l &= \pm 0, d_1 d_2 \cdots d_n \cdots \times b^l \\ &= \pm (d_1 \times b^{-1} + d_2 \times b^{-2} + \cdots + d_n \times b^{-n} + \cdots) \times b^l \end{aligned}$$

où

- $m = 0, d_1 d_2 \cdots d_n \cdots$ est la **mantisse**, par convention on prendra $m < 1$.
- b est la **base**, $b \in \mathbb{N}$,
- l est l'**exposant**, $l \in \mathbb{R}$.

De plus, les entiers d_i dans l'écriture de la mantisse vérifient :

- $0 \leq d_i \leq b-1, \ i = 2, 3, \cdots$
- $0 < d_1 \leq (b-1)$, cela permet d'assurer l'unicité de la représentation. On parlera de virgule flottante à **mantisse normalisée**.

Dans ce cas, on peut montrer

$$\frac{1}{b} \leq m < 1$$

à noter que l'écriture de 0 devient une exception (car la mantisse ne s'annule jamais !).

Remarques :

- Le choix de la base b est arbitraire, cependant il y a des choix naturels :
  - b=10, base décimale : utilisée par les humains,
  - b=2, base binaire, ou b=16, base hexadécimale : utilisées par les ordinateurs.

Représentation en base binaire :

Soit N un entier positif $(N)_{10} = (d_n \cdots d_2 d_1 d_0)_2$

Exemple : N=30, $(30)_{10} = (11110)_2$

$$\begin{aligned} 30 \div 2 = 15 \quad & reste \quad 0 \longrightarrow d_0 \quad = 0 \\ 15 \div 2 = 7 \quad & reste \quad 1 \longrightarrow d_1 \quad = 1 \\ 7 \div 2 = 3 \quad & reste \quad 1 \longrightarrow d_2 \quad = 1 \\ 3 \div 2 = 1 \quad & reste \quad 1 \longrightarrow d_3 \quad = 1 \\ 1 \div 2 = 0 \quad & reste \quad 1 \longrightarrow d_4 \quad = 1. \end{aligned}$$

Soit f une fraction décimale $(f)_{10} = (0, d_1 d_2 \cdots d_n)_2$

Exemple : f = 0.0625, $(0.0625)_{10} = (0, 0001)_2$

$$\begin{aligned} 0.0625 \times 2 = 0 + 0.125 \quad & \longrightarrow d_1 = 0 \\ 0.125 \times 2 = 0 + 0.25 \quad & \longrightarrow d_2 = 0 \\ 0.25 \times 2 = 0 + 0.5 \quad & \longrightarrow d_3 = 0 \\ 0.5 \times 2 = 1 + 0. \quad & \longrightarrow d_4 = 1. \end{aligned}$$

En conclusion $(30, 0625)_{10} = (11110, 0001)_2$

La représentation en virgule flottante de en base 2 : $30.0625 = 0,11110001 \times 2^5$

Représentation en norme : IEEE-754 sur 32 bits

On sait que $2^{8-1} - 1 = 127$

$$\begin{aligned}(Y)_{10} &= (d_1 d_2 d_3 \cdots d_{32})_2 \\ &= (-1)^{(d_1)_2} \quad 2^{(d_2 \cdots d_9)_2} 2^{-127} \quad (1, d_{10} \cdots d_{32})_2\end{aligned}$$

Exemple :

$$\begin{aligned}(30.0625)_{10} &= (11110, 0001)_2 \\ &= \mathbf{1}, 11100001 \times 2^4 = 1, d_{10} \cdots 0\end{aligned}$$

Le chiffre **1** dans la dernière ligne n'est pas pris en compte, on complète par des zéros pour avoir 23 bis après la vrigule.

Maintenant on cherche la partie exposant :

$x - 127 = 4 \longrightarrow x = 131 = (10000011)_2$

La représentation sera : 0 10000011 11100001000000000000000

Représentation en norme : IEEE-754 sur 64 bits

On sait que $2^{11-1} - 1 = 1023$

$$\begin{aligned}(Y)_{10} &= (d_1 d_2 d_3 \cdots d_{64})_2 \\ &= (-1)^{(d_1)_2} \quad 2^{(d_2 \cdots d_{12})_2} 2^{-1023} \quad (1, d_{13} \cdots d_{64})_2\end{aligned}$$

Remarques (suite) :

- **Quelle que soit la base choisie**, les nombres irrationnels $(\pi, \sqrt{2}, \cdots)$ auront toujours une mantisse de longueur infinie,
- L'utilisation de mantisse infinie sera bien évidemment impossible sur un ordinateur, où le nombre de bits pour représenter un nombre réel est limité. Ainsi, on recourt à la troncature ou à l'arrondi pour réduire la mantisse à N chiffres. On aura donc

$$m = d_1 \times b^{-1} + d_2 \times b^{-2} + \cdots + d_N \times b^{-N}$$

La représentation de certains réels sur ordinateur sera donc erronée. De plus les opérations élémentaires utilisant ces nombres seront également erronées.

Troncature et arrondi :

- Troncature à N chiffres : on coupe la mantisse au-delà de la position N.
- Arrondi à N chiffres : On ajoute (5 en décimal, 1 en binaire) à la position $N+1$, puis on fait une troncature à N chiffres.

Definition

"Précision machine" On appelle précision machine (notée ϵ_m) la plus grande erreur relative commise en représentant un nombre réel sur ordinateur en utilisant la troncature :

$$\epsilon_m = \max_x \frac{\Delta x}{|x|} = \max_x E_r(x).$$

En utilisant l'arrondi, la plus grande erreur relative est $\frac{\epsilon_m}{2}$.

Il est possible de montrer que l'on a

$$\epsilon_m \leq b^{1-N}$$

N étant le nombre de chiffres dans la mantisse.

Preuve :
$x = \pm 0, d_1 d_2 d_3 \cdots d_n d_{n+1} \cdots \times b^l$

$$\begin{aligned}\Delta x &= 0, 0 \cdots 0 d_{n+1} \cdots \times b^l \\ &\leq 0, d_{n+1} \cdots \times b^{l-n} \\ &\leq 0, (b-1)(b-1) \cdots \times b^{l-n}.\end{aligned}$$

Alors

$$\begin{aligned}E_r(x) = \frac{\Delta x}{|x|} &\leq \frac{0, (b-1)(b-1) \cdots \times b^{l-n}}{0, d_1 d_2 \cdots \times b^l} \\ &\leq \frac{0, (b-1)(b-1) \cdots \times b^{l-n}}{0, 1 \cdots \times b^l} \\ &\leq b \times b^{-n} = b^{1-n}\end{aligned}$$

On a utlisé $\frac{1}{b} \leq m < 1$ pour conclure la dernière ligne de la démonstration

Remarques :

- En pratique, on ne fera pas de distinctions entre ϵ_m et b^{1-N},
- Norme IEEE-754 : standardisation de la représentation. Garantis l'uniformité du comportement entre les ordinateurs. Précision machine :

  - $2^{1-24} = 0,119 \times 10^{-6}$ en simple précision
  - $2^{1-53} = 0,222 \times 10^{-15}$ en double précision

Conclusion : En général les nombres ne seront plus représentés exactement. Il y aura des erreurs dans le résultat d'opérations arithmétiques de base !

Definition

"Notation flottante" Soit $x \in \mathbb{R}$. On note $fl(x)$ sa **représentation en virgule flottante décimal** à n chiffres

$$fl(x) = \pm 0, d_1 d_2 d_3 \cdots d_n \times 10^l$$

le dernier chiffre d_n dépend du procédé retenu pour éliminer les derniers chiffres, à savoir la troncature ou l'arrondi. **On utilisera de préférence l'arrondi**.

On utilise la base décimale, cependant les effets décrits par la suite valent également pour les autres bases !

Principe de fonctionnement des opérations élémentaires : On doit représenter les deux opérandes en notation flottante, effectuer l'opération de la façon habituelle et exprimer le résultat en notation flottante.

Opérations élémentaires :

- $x + y \longrightarrow fl(fl(x) + fl(y))$
- $x - y \longrightarrow fl(fl(x) - fl(y))$
- $x \div y \longrightarrow fl(fl(x) \div fl(y))$
- $x \times y \longrightarrow fl(fl(x) \times fl(y))$

Remarques :

- $+/-$: on devra faire un «décalage » de la mantisse lorsque les exposants ne sont pas les mêmes. Cela impliquera d'ajouter des zéros dans le nombre le plus petit (pour ramener son exposant au niveau du plus grand nombre).
- $\times/\div$: la loi des exposants permet de faire la somme (différence) des exposants et uniquement le produit (la division) des mantisses pour obtenir le résultat.

Exemples1 : dans ces exemples on travaille avec une arithmétique flottante avec n = 3 chiffres dans la mantisse, on obtient d'une part :

$$\begin{aligned}
122 \times (333 + 695) &= fl[(0, 122.10^3) \times fl(0, 333.10^3 + 0, 695.10^3)] \\
&= fl[(0, 122.10^3) \times fl(1, 028.10^3)] \\
&= fl[(0, 122.10^3) \times (0, 103.10^4)] \\
&= fl(0, 012566.10^7) \\
&= 0, 126 \times 10^6
\end{aligned}$$

par contre

$$\begin{aligned}
(122 \times 333) + (122 \times 695) &= fl[fl((0, 122.10^3) \times (0, 333.10^3)) + fl((0, 122.10^3) \times (0, \\
&= fl[fl(0, 040626.10^6) + fl(0, 08479.10^6)] \\
&= fl[(0, 406.10^5) + (0, 848.10^5)] \\
&= fl(1, 254.10^5) \\
&= 0, 125.10^6
\end{aligned}$$

On remarque constate donc une petite différence entre les deux résultats, alors ces opérations ne sont pas équivalentes en arithmétique flottante.

Exemples2 : Additionner deux nombres dont les ordres de grandeur sont très différents :

$$\begin{aligned}(0,400.10^4)+(0,100.10^{-2}) &= fl(0,400.10^4+0,100.10^{-2})\\ &= fl(0,400.10^4+0,0000001.10^4)\\ &= fl(0,4000001.10^4)\\ &= 0,400.10^4\end{aligned}$$

On remarque que le petit nombre disparaît devant le plus grand.

attention

"Opérations risquées"

- *Le décalage de la mantisse peut se révéler* ***très dangereux si on additionne des nombres dont les ordres de grandeur sont très différents****.*
- *La troncature ou l'arrondi pouvant se faire sur certains termes intermédiaires,* ***l'associativité et la distributivité*** *n'existe plus :* ***l'ordre des opérations*** *devient important pour le résultat.*
- *Il peut être dangereux de soustraire des* ***nombres très proches*** *: perte de précision (chiffres significatifs).*

Réelle source de problèmes en pratique !

Suivant le problème, il existe certaines méthodes de contournement : additionner les termes en les plaçant par ordre croissant, ...

Conclusion :

- Les opérations élémentaires sont sources d'erreurs,
- Pas de solution miracle répondant à tous les problèmes énoncés auparavant !
- On essayera de **minimiser le nombre d'opérations**. à ce titre, le nombre d'opérations élémentaires en virgule flottante sert souvent de point de comparaison entre différents algorithmes.

Ex : **algorithme de Horner** pour l'évaluation d'un polynôme.

Algorithme de Horner : multiplications imbriquées.
Soit le polynôme p de degré n, $p(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n$.
En regroupant judicieusement les termes, on peut écrire :

$$\begin{aligned} p(x) &= a_0 + a_1x + a_2x^2 + a_3x^3 + \cdots + a_nx^n \\ &= a_0 + x(a_1 + a_2x + a_3x^2 + \cdots + a_{n-1}x^{n-2} + a_nx^{n-1}) \\ &= a_0 + x(a_1 + x(a_2 + a_3x + \cdots + a_{n-1}x^{n-3} + a_nx^{n-2})) \\ &= \cdots \\ &= a_0 + x(a_1 + x(a_2 + x(a_3 + \cdots + x(a_{n-1} + a_nx)\cdots))) \end{aligned}$$

Approche directe : au plus $\frac{n(n+1)}{2} + n$ opérations élémentaires $(\times/+)$.
Horner : au plus $2n$ opérations élémentaires.

Exemple :
Soit le polynôme p de degré n, $p(x) = 6 + 1 * x + 2 * x^2 + 3 * x^3 + 4 * x^4$.
Le polynôme de Horner sera : $p(x) = 6 + x(1 + x(2 + x(3 + 4x)))$
Approche directe : au plus $\frac{4(4+1)}{2} + 4 = 14$ opérations élémentaires, 10 multiplication et 4 addition
Horner : au plus $2 * 4 = 8$ opérations élémentaires, 4 multiplication et 4 addition.

Data:

- Les coefficients a_i d'un polynôme p de degré n
- une abscisse x

Result: la variable r qui contiendra la valeur du polynôme p évaluée en x

$r = a_n$;
for $i = n, n-1, \cdots, 2, 1$ **do**
| $r = a_{i-1} + xr$
end
$p(x) = r$;

Algorithm 1: Algorithme de Horner, pseudo-code

Tout cela concerne l'erreur de représentation. Qu'en est-il de l'erreur de troncature ?

$\rightarrow$ Développement de Taylor

Chapitre 1 : Analyse d'erreurs : Erreur de troncature (Taylor-Lagrange)

- Quel est le polynôme de degré n, noté $\boldsymbol{P_n(x)}$, qui donne la meilleure approximation d'une **fonction f donnée**, cela **au voisinage d'un point x_0, également donné** ?

$\rightarrow$ Le polynôme (ou formule) de Taylor nous donne une méthode de construction des polynômes $P_n(x)$, **lorsque la fonction f est suffisamment dérivable.**

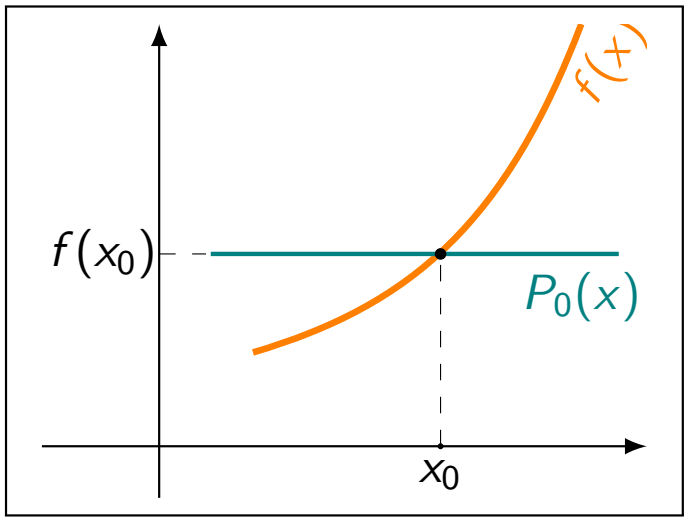
Definition

"Polynôme de Taylor" Le polynôme de Taylor de degré n de la fonction $f(x)$, autour du point x_0 est défini par

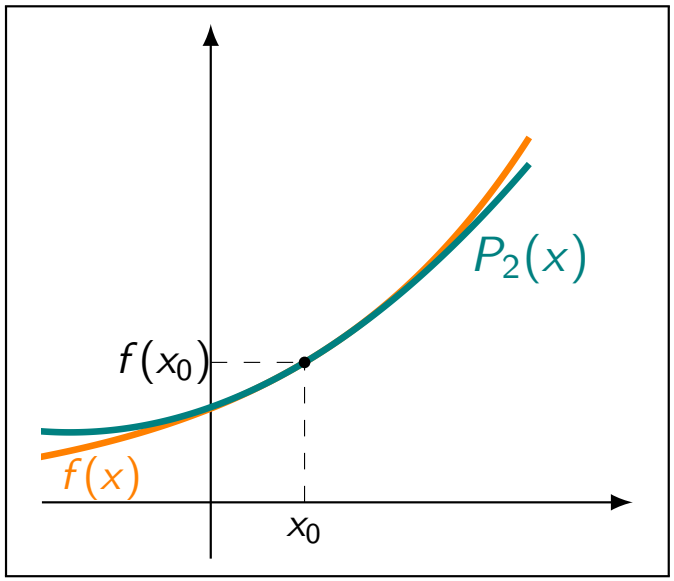
$$\begin{aligned} P_n(x) =& f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \\ & \frac{f'''(x_0)}{3!}(x - x_0)^3 + \cdots + \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n \\ =& \sum_{i=0}^{n} \frac{f^{(i)}(x_0)}{i!}(x - x_0)^i \end{aligned}$$

Note : la fonction f doit être n fois dérivable en x_0

Chapitre 1 : Analyse d'erreurs : Erreur de troncature (Taylor-Lagrange)



Chapitre 1 : Analyse d'erreurs : Erreur de troncature (Taylor-Lagrange)



- $n = 2$

$$P_2(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2}(x - x_0)^2.$$

Cela signifie que l'on approche f par une quadratique, un polynôme de degré 2. Approcher une fonction f par un polynôme de Taylor P_n va induire une erreur,

donnée par l'expression

Chapitre 1 : Analyse d'erreurs : Erreur de troncature (Taylor-Lagrange)

Theorem

"Développement de Taylor-Lagrange" Soit f une fonction ***$n+1$ fois dérivable*** *autour de x_0, et $P_n(x)$ le polynôme de Taylor de degré n autour de x_0. On a* ***l'égalité*** *suivante*

$$f(x) = P_n(x) + R_n(x)$$

R_n est «l'écart» entre f et P_n, défini par

$$R_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!}(x - x_0)^{n+1}$$

où ξ_x est un nombre réel, dépendant de x, compris entre x et x_0, i.e

- $x \leq \xi_x \leq x_0$ *si* $x < x_0$
- $x_0 \leq \xi_x \leq x$ *si* $x > x_0$

L'erreur de troncature est ainsi définie par

$$|f(x) - P_n(x)| = |R_n(x)| = \left|\frac{f^{(n+1)}(\xi_x)}{(n+1)!}(x - x_0)^{n+1}\right| = \left|\frac{f^{(n+1)}(\xi_x)}{(n+1)!}\right| \left|(x - x_0)^{n+1}\right|$$

Chapitre 1 : Analyse d'erreurs : Erreur de troncature (Taylor-Lagrange)

Remark

- *à n fixé, l'erreur augmente si x s'éloigne de x_0 et diminue si x se rapproche de x_0,*
- *il est donc préférable d'avoir x_0 proche des points où l'on souhaite évaluer la fonction f ...*

On cherche à évaluer l'erreur de troncature

$$|R_n(x)| = |f(x) - P_n(x)| = \left| \frac{f^{(n+1)}(\xi_x)}{(n+1)!}(x - x_0)^{n+1} \right|$$

Cependant ξ_x n'est pas connu ! ! ! On sait seulement qu'il existe et qu'il dépend de n, x, x_0, f.

Afin d'estimer l'erreur, le but du jeu va donc être de chercher à se «débarrasser» de la variable ξ_x.

Chapitre 1 : Analyse d'erreurs : Erreur de troncature (Taylor-Lagrange)

Pour cela, on va chercher à majorer le terme $\left|f^{(n+1)}(\xi_x)\right|$, *i.e* trouver $M \in \mathbb{R}^+$ tel que

$$\left|f^{(n+1)}(\xi_x)\right| \leq M$$

On aura ainsi $|R_n(x)| = |f(x) - P_n(x)| \leq \frac{M}{(n+1)!}\left|(x-x_0)^{n+1}\right|$. On contrôle ainsi le terme d'erreur. **Formulation en h du développement de Taylor :**

On utilise souvent une autre forme du développement de Taylor, obtenu en effectuant le changement de variable $h = x - x_0$. On obtient ainsi

$$f(x) = f(x_0 + h) = P_n(x_0 + h) + R_n(x_0 + h) = P_n(h) + R_n(h)$$

avec

$$P_n(h) = f(x_0) + f'(x_0)h + \frac{f''(x_0)}{2!}h^2 + \frac{f'''(x_0)}{3!}h^3 + \cdots + \frac{f^{(n)}(x_0)}{n!}h^n$$

$$R_n(h) = \frac{f^{(n+1)}(\xi_h)}{(n+1)!}h^{n+1}$$

Chapitre 1 : Analyse d'erreurs : Erreur de troncature (Taylor-Lagrange)

ξ_h compris entre x_0 et $x_0 + h$:

- Si $h > 0 \rightarrow \xi_h \in [x_0, x_0 + h] \iff x_0 \leq \xi_h \leq x_0 + h$
- Si $h < 0 \rightarrow \xi_h \in [x_0 + h, x_0] \iff x_0 + h \leq \xi_h \leq x_0$

attention

Il faut savoir manipuler les deux formulations du développement de Taylor, et ne pas les mélanger !

Avec la formulation en h on cherchera ainsi une majoration du terme d'erreur de la forme

$$|R_n(h)| = \left| \frac{f^{(n+1)}(\xi_h)}{(n+1)!} \right| |h|^{n+1} \leq \frac{M}{(n+1)!} |h|^{n+1}$$

Chapitre 1 : Analyse d'erreurs : Erreur de troncature (Taylor-Lagrange)

Definition

"Notation de Landau - grand ordre" Une fonction $e(h)$ est un grand ordre de h^n au voisinage de 0, on notera $e(h) = O(h^n)$ s'il existe une constante C telle que

$$|e(h)| \leq Ch^n$$

Remarques :

- **Lorsque h est assez petit**, la fonction $O(h^n)$ décroît comme Ch^n,
- Plus n est grand, plus la décroissante est rapide,
- Une fonction $O(h^p)$ est obligatoirement $O(h^m)$ pour $m < p$.
- Comment déterminer le grand ordre d'une fonction ? Pour h petit,

$$e\left(\frac{h}{2}\right) \approx C\left(\frac{h}{2}\right)^n = C\frac{h^n}{2^n} \Rightarrow \frac{e(h)}{e\left(\frac{h}{2}\right)} \approx 2^n$$

à titre d'exemple, si $e(h) = O(h^n)$ représente une erreur, en divisant h par 2, l'erreur sera divisée par 2^n.

Chapitre 1 : Analyse d'erreurs : Erreur de troncature (Taylor-Lagrange)

Definition

"Ordre d'une approximation" Une approximation dont le **terme d'erreur** est un grand ordre de h^n ($O(h^n)$) est dite approximation **d'ordre** n.

• Suivant cette définition, le polynôme de Taylor de degré n sera généralement, mais pas toujours, une approximation d'ordre $n+1$, *i.e*

$$R_n(x) = O((x-x_0)^{n+1}) \text{ ou } R_n(h) = O(h^{n+1}).$$

Un contre exemple est le développement de Taylor d'ordre 5 de $f(x) = \sin(x)$ autour de 0 : il suffit de calculer le polynôme de Taylor de degré 3 pour avoir une approximation d'ordre 5.

• NE PAS CONFONDRE **le degré** du polynôme P_n **avec l'ordre** du développement de Taylor

- Le degré est la puissance de h (ou $(x-x_0)$) la plus grande dans le polynôme P_n,
- L'ordre est la puissance de h (ou $(x-x_0)$) intervenant dans le terme d'erreur R_n, et sert à indiquer la qualité de l'approximation.

Chapitre 1 : Analyse d'erreurs : Erreur de troncature (Taylor-Lagrange)

Avec la notion de grand ordre, il est donc possible d'écrire les développements de Taylor de la façon suivante ($n = 2$ pour illustrer) :

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + O((x - x_0)^3)$$

Formulation en h :

$$f(x_0 + h) = f(x_0) + f'(x_0)h + \frac{f''(x_0)}{2!}h^2 + O(h^3)$$

Cette dernière expression est pratique pour évaluer facilement la fonction f en un point proche de x_0.

Chapitre 1 : Analyse d'erreurs : Erreur de troncature (Taylor-Lagrange)

Exemple :
On considère le développement de Taylor la fonction sin autour de x_0 obtenu en effectuant le changement de variable $h = x - x_0$. On obtient ainsi

$$\sin(x) = \sin(x_0 + h) = \sin(h) = h - \frac{h^3}{3!} + \frac{\cos(\xi_h)}{5!}h^5$$

si $h = 0.1$ On obtient $\sin(0.1) = 0.1 - \frac{0.1^3}{3!} = 0.099833333$, avec une borne supérieure de $e(0.1) = 0.08333 * 10^{-7}$

si $h = 0.2$ On obtient $\sin(0.2) = 0.2 - \frac{0.2^3}{3!} = 0.1986666667$, avec une borne supérieure de $e(0.2) = 0.2664 * 10^{-5}$

en faisant la fraction des deux borne supérieure, on obtient $\frac{0.2664*10^{-5}}{0.08333*10^{-7}} = 31.97 = 2^5$.

Cet Exemple nous montre que le polynôme de taylor de degré 3 de la fonction $\sin(x)$ est ien d'ordre 5.

$e\left(\frac{h}{2}\right) \approx C\left(\frac{h}{2}\right)^n = C\frac{h^n}{2^n} \Rightarrow \frac{e(0.2)}{e\left(\frac{0.2}{2}\right)} \approx 2^n$

Chapitre 1 : Analyse d'erreurs : Erreur de troncature (Taylor-Lagrange)

Propagation d'erreur :
Supposons connu une certaine approximation x^* (un paramètre physique par exemple), avec Δx connu. On souhaite évaluer la précision du calcul $f(x^*)$. L'erreur absolue est définie par

$$\Delta f = |f(x) - f(x^*)| = |f(x^* \pm \Delta x) - f(x^*)|$$

Mais par développement de Taylor de f en x^*, on a

$$f(x^* \pm \Delta x) = f(x^*) \pm f'(x^*)\Delta x + O((\Delta x)^2)$$

On a ainsi

$$\Delta f \approx |f'(x^*)|\Delta x$$

à partir de l'erreur absolue sur x^*, on peut donc estimer l'erreur absolue sur $f(x^*)$

Chapitre 1 : Analyse d'erreurs : Erreur de troncature (Taylor-Lagrange)

Soit $f(x, y)$ une fonction de deux variables, à valeurs réelles que l'on suppose suffisamment différentiable. Le développement de Taylor d'ordre 2 de la fonction $f(x, y)$ autour du point (x_0, y_0) est défini par

$$f(x_0 + h_1, y_0 + h_2) = f(x_0, y_0) + \frac{\partial f}{\partial x}(x_0, y_0)h_1 + \frac{\partial f}{\partial y}(x_0, y_0)h_2 + O(h_1^2) + O(h_2^2),$$

Pour une fonction de trois variables $f(x, y, z)$, on aura autour de (x_0, y_0, z_0)

$$\begin{aligned} f(x_0 + h_1, y_0 + h_2, z_0 + h_3) = f(x_0, y_0, z_0) &+ \frac{\partial f}{\partial x}(x_0, y_0, z_0)h_1 \\ &+ \frac{\partial f}{\partial y}(x_0, y_0, z_0)h_2 + \frac{\partial f}{\partial z}(x_0, y_0, z_0)h_3 \\ &+ O(h_1^2) + O(h_2^2) + O(h_3^2) \end{aligned}$$

Dans le cadre de ce cours, on n'explicitera pas le terme d'erreur pour les fonctions de plusieurs variables.

Chapitre 1 : Analyse d'erreurs : Erreur de troncature (Taylor-Lagrange)

Cela nous fournit une formule pour la propagation d'erreur dans le cadre de fonctions à plusieurs variables

$$(x, y, z) = (x^*, y^*, z^*) \pm (\Delta x, \Delta y, \Delta z)$$

$$\begin{aligned}\Delta f = |f(x, y, z) - f(x^*, y^*, z^*)| \simeq & \left|\frac{\partial f}{\partial x}(x^*, y^*, z^*)\right| \Delta x \\ & + \left|\frac{\partial f}{\partial y}(x^*, y^*, z^*)\right| \Delta y \\ & + \left|\frac{\partial f}{\partial z}(x^*, y^*, z^*)\right| \Delta z\end{aligned}$$

Propagation d'erreur absolue pour les opérations élémentaires :

$$\begin{aligned}
f(x, y) = x + y \quad &\longrightarrow \quad \Delta f \simeq |\Delta x| + |\Delta y| \\
f(x, y) = x - y \quad &\longrightarrow \quad \Delta f \simeq |\Delta x| + |\Delta y| \\
f(x, y) = x \times y \quad &\longrightarrow \quad \Delta f \simeq |y^*||\Delta x| + |x^*||\Delta y| \\
f(x, y) = x \div y \quad &\longrightarrow \quad \Delta f \simeq \frac{|y^*||\Delta x| + |x^*||\Delta y|}{\quad}
\end{aligned}$$

Conclusion chapitre 1, je dois savoir faire :

- *Calculer/estimer l'erreur absolue et relative d'un nombre,*
- *établir les chiffres significatifs partant de l'erreur absolue et réciproquement,*
- *Faire une représentation en virgule flottante avec mantisse à n chiffres par troncature ou par arrondi,*
- *Normaliser une mantisse et décaler la mantisse le cas échéant,*
- *Faire des opérations élémentaires en virgule flottante à n chiffres,*
- *Reconnaître les opérations dangereuses et comprendre l'importance de l'ordre dans les opérations élémentaires,*
- *Utiliser la méthode de Horner,*
- *Construire un polynôme de Taylor de degré n ainsi que son terme d'erreur,*
- *Estimer le reste, approximer l'erreur absolue et obtenir le nombre de chiffres significatifs de mon approximation,*
- *Comprendre et savoir calculer l'ordre d'une méthode,*
- *Savoir calculer la propagation d'erreur pour une fonction d'une ou de plusieurs variables et ainsi calculer l'erreur/chiffres significatifs d'une évaluation de fonction.*